Task 9: Synchronizing with Effects in React

Summary

In this task, I learned how to use the useEffect() Hook to perform side effects in React components. Unlike rendering, which should only return JSX, effects are used to synchronize React components with external systems such as APIs, timers, browser events, local storage, and the DOM. I understood that useEffect() runs after a component has been rendered, making it the correct place to perform operations that interact with the outside world.

I learned how the dependency array controls when an effect executes. An empty dependency array ([]) makes the effect run only once after the initial render, while providing specific dependencies causes the effect to run only when those values change. If no dependency array is provided, the effect runs after every render. Understanding dependencies helped me avoid unnecessary executions and improve application performance.

Another important concept I learned was the cleanup function. Some effects, such as event listeners, timers, intervals, or subscriptions, continue running even after a component is removed from the screen. By returning a cleanup function from useEffect(), these resources can be properly cleaned up, preventing memory leaks and unexpected behavior in the application.

I also understood that not every operation requires an effect. Calculating values from props or state, updating state based on user input, or rendering UI should be handled directly during rendering instead of using useEffect(). Effects should only be used when synchronizing with systems outside React. This helps keep components simpler, more efficient, and easier to maintain.

Finally, I learned the complete lifecycle of an effect—when it starts, when it re-runs because dependencies change, and when it cleans up before running again or when the component unmounts. This knowledge helped me understand how React manages side effects and keeps components synchronized with external data and services.

Learning Outcome

After completing this task, I understood how to use the useEffect() Hook to handle side effects in React applications, how dependency arrays control effect execution, how cleanup functions prevent memory leaks, and when effects should or should not be used. These concepts are essential for working with APIs, timers, browser events, and other external systems in modern React applications.

Task 10: Managing State in React

Summary

In this task, I learned how to manage and organize state efficiently in React applications as they become larger and more complex. I understood that React follows a state-driven approach, where the user interface automatically updates whenever the component's state changes instead of directly manipulating the DOM. User interactions such as typing, clicking, or submitting forms update the state, and React re-renders the UI accordingly.

I learned the importance of designing a proper state structure by avoiding redundant or duplicate state. Instead of storing values that can be calculated from existing state, such as a user's full name, those values should be derived during rendering. This reduces bugs, simplifies the code, and makes components easier to maintain.

Another important concept I learned was lifting state up. When multiple components need to share the same data, the shared state should be moved to their closest common parent component. The parent then passes the required data and event handler functions to its child components through props. This ensures that all related components remain synchronized and use a single source of truth.

I also learned how React preserves component state during re-rendering and how the key prop can be used to reset a component's state when needed. This is useful in situations where switching between different data should display a fresh component instead of preserving the previous state, such as changing chat recipients or form data.

As applications grow, managing state with multiple useState() hooks can become difficult. I learned how the useReducer() Hook helps organize complex state update logic by moving it into a separate reducer function. Instead of directly updating state, components dispatch actions, making the code cleaner, easier to debug, and more scalable.

Finally, I learned about the Context API, which solves the problem of prop drilling. Instead of passing props through many intermediate components, Context allows data to be shared directly with deeply nested components. I also understood how combining useReducer() with Context provides a powerful and scalable state management solution for large React applications by centralizing both state and update logic.

Learning Outcome

After completing this task, I understood how to structure and manage state efficiently in React, avoid redundant state, share state between components by lifting it up, preserve or reset component state using the key prop, organize complex state logic with useReducer(), and share data across deeply nested components using the Context API. These concepts are fundamental for building scalable, maintainable, and efficient React applications.

⇨ **GITHUB REPO LINK:**

https://github.com/yashrathod910/15D_Internship/tree/main/DAY6